Collection of Pseudo-Code definitions for the Tensor Omics Implementation

Asis Hallab, “el Bobito”

May 7, 2025

1 F42 Core Sorting Module Pseudocode

1.1 Module Overview

The sorting module provides type-specific, in-place sorting routines for 1D arrays of integers, real numbers, and character sequences. It uses Fortran `interface` blocks to overload a common sort function name across types. All working memory is preallocated and passed explicitly.

1.2 Type Definitions

- No new types are defined.
- Only standard Fortran arrays are used.

1.3 Public Interface

- `interface sort_array`
  - `module procedure sort_integer, sort_real, sort_character`

Each sort routine works in-place and expects preallocated working memory.

1.4 Function Signatures

- `subroutine sort_integer(array, work)`
  - `integer, dimension(n), intent(inout) :: array`
  - `integer, dimension(n), intent(inout) :: work` (temporary work array)
- `subroutine sort_real(array, work)`
  - `real, dimension(n), intent(inout) :: array`
  - `integer, dimension(n), intent(inout) :: work`
- `subroutine sort_character(array, work)`
  - `character(len=*), dimension(n), intent(inout) :: array`
  - `integer, dimension(n), intent(inout) :: work`

1.5 Algorithm: General Structure (All Types)

1. Initialize permutation array:
$$\text{work}(i) = i \quad \text{for} \quad i = 1, \ldots, n$$
2. Sort the permutation array `work` based on the corresponding values in `array`.
   - Use insertion sort (small n) or quicksort (large n).
3. Rearrange `array` according to the sorted permutation:
   - Use a cycle leader algorithm to avoid extra memory allocation.

1.6 Details for Each Type

- **Integer**: Compare `array(i)` and `array(j)` numerically.
- **Real**: Compare `array(i)` and `array(j)` numerically.
- **Character**: Compare `array(i)` and `array(j)` lexicographically.

1.7 Memory Requirements

- For sorting an array of size n:
  - One integer work array of size n is needed.

1.8 Algorithm: Non-Recursive Quicksort for Permutation Array

To comply with F42 and WebAssembly restrictions, Quicksort is implemented without recursion, using an explicit manual stack structure for subarray boundaries.

1.8.1 Procedure: `quicksort_nonrecursive`

- `subroutine quicksort_nonrecursive(array, perm, n, stack_left, stack_right)`
  - `type depends on context :: array` (integer, real, or character array)
  - `integer, dimension(n), intent(inout) :: perm` (permutation array)
  - `integer, intent(in) :: n` (size of array)
  - `integer, dimension(:), intent(inout) :: stack_left, stack_right` (manual stack arrays)

1.8.2 Working Memory Requirements

- `stack_left` and `stack_right`: integer arrays of length at least $\log_2(n) + 10$ (safe overestimate).

1.8.3 Steps

1. Initialize:
   - Set `stack_ptr = 1`.
   - Push initial full range: `stack_left(1) = 1`, `stack_right(1) = n`.
2. Main loop:
   - While `stack_ptr > 0`:
     (a) Pop:
     - Set `left = stack_left(stack_ptr)`, `right = stack_right(stack_ptr)`.

- Decrement `stack_ptr`.

(b) If `left` $\geq$ `right`, continue to next loop iteration.

(c) Select pivot:
- Set `pivot_index = (left + right) / 2`.
- Set `pivot_value = array(perm(pivot_index))`.

(d) Partition:
- Initialize `i = left`, `j = right`.
- While `i` $\leq$ `j`:
  - Increment `i` while `array(perm(i)) < pivot_value`.
  - Decrement `j` while `array(perm(j)) > pivot_value`.
  - If `i` $\leq$ `j`:
    - Swap `perm(i)` and `perm(j)`.
    - Increment `i`, decrement `j`.

(e) Push new ranges onto the stack:
- If `left < j`:
  - Increment `stack_ptr`.
  - Set `stack_left(stack_ptr) = left`, `stack_right(stack_ptr) = j`.
- If `i < right`:
  - Increment `stack_ptr`.
  - Set `stack_left(stack_ptr) = i`, `stack_right(stack_ptr) = right`.

1.8.4 Notes

- No recursion: stack is manually managed.
- Memory predictable: all working memory passed as arguments.
- Stateless: no hidden or module-wide variables.
- Safe for WebAssembly compilation and fully F42 compliant.

2 Unit Tests for F42 Core Sorting Module

The following unit tests verify correctness, robustness, and F42 compliance of the non-recursive sorting routines for integer, real, and character arrays. Each test assumes that all required working memory is preallocated before calling sort procedures.

2.1 Test 1: Sort Ascending Integer Array

- Input:
  - `array` = [5, 2, 9, 1, 6]
  - `work` = preallocated integer array of size 5
- Action:
  - Call `sort_array(array, work)`.
- Expected:
  - `array` = [1, 2, 5, 6, 9]

2.2 Test 2: Sort Descending Real Array

- Input:
  - `array` = [3.5, 2.2, 8.8, 1.1, 7.7]
  - `work` = preallocated integer array of size 5
- Action:
  - Call `sort_array(array, work)`.
- Expected:
  - `array` = [1.1, 2.2, 3.5, 7.7, 8.8]

2.3 Test 3: Sort Random Character Array

- Input:
  - `array` = [”dog”, ”apple”, ”zebra”, ”cat”, ”bird”]
  - `work` = preallocated integer array of size 5
- Action:
  - Call `sort_array(array, work)`.
- Expected:
  - `array` = [”apple”, ”bird”, ”cat”, ”dog”, ”zebra”]

2.4 Test 4: Stability on Already Sorted Arrays

- Input:
  - `array` = [1, 2, 3, 4, 5]
  - `work` = preallocated integer array of size 5
- Action:
  - Call `sort_array(array, work)`.
- Expected:
  - `array` remains [1, 2, 3, 4, 5]

2.5 Test 5: Behavior on Empty Array

- Input:
  - `array` = []
  - `work` = preallocated integer array of size 0
- Action:
  - Call `sort_array(array, work)`.
- Expected:
  - No error or crash; operation completes trivially.

2.6 Test 6: Correctness of Sorting After Random Shuffles

- Input:
  - Generate random permutation of integers [1,2,3,...,1000].
  - `array` = shuffled integers
  - `work` = preallocated integer array of size 1000
- Action:
  - Call `sort_array(array, work)`.
- Expected:
  - `array` = [1, 2, 3, ..., 1000]

3 Generalized F42-Compliant LOESS Smoothing Routine

3.1 Purpose

This routine provides a reusable LOESS (Locally Weighted Scatterplot Smoothing) kernel smoother for scalar or vector data in Tensor Omics. Inspired by R's `loess(formula, data)`, this version uses strict F42 conventions:

- Stateless: No persistent state.
- Allocation-free: All inputs, outputs, and workspaces passed explicitly.
- Modular: Works on scalars, vectors, or matrix-shaped fields.
- Efficient: Supports filtering via index arrays and optional kernel cutoff.

3.2 Function Signature

```
subroutine loess_smooth(
    n_total,                        ! total entries in the source arrays
    n_target,                       ! number of smoothing query targets

    x_ref(n_total),                 ! reference coordinate array (e.g. mean values)
    y_ref(d, n_total),              ! values at x_ref (scalars: d=1, vectors: d > 1)

    indices_used(n_target),        ! index array specifying which entries to use
    x_query(n_target),             ! query locations for LOESS evaluation
    y_out(d, n_target),            ! smoothed output at query locations

    kernel_sigma,                  ! width of Gaussian kernel
    kernel_cutoff,                 ! cutoff radius (e.g. 3.0 = truncate at 3sigma)

    workspace_weights(n_total),    ! transient workspace (reused per query point)
    workspace_values(d, n_total)   ! transient workspace for weighted values
)
```

3.3 F42 Pseudocode Implementation

```
for q = 1 to n_target:

    query_x = x_query(q)
    sum_weights = 0.0

    ! zero out y_out(:, q)
    for k = 1 to d:
        y_out(k, q) = 0.0

    for i = 1 to n_total:

        idx = indices_used(i)
        ref_x = x_ref(idx)
        delta = abs(query_x - ref_x)

        if delta > kernel_cutoff * kernel_sigma:
            cycle

        weight = exp( - (delta / kernel_sigma)**2 )
        sum_weights += weight

        ! Accumulate weighted value
        for k = 1 to d:
            y_out(k, q) += weight * y_ref(k, idx)

    ! Normalize if possible
    if sum_weights > 0.0:
        for k = 1 to d:
            y_out(k, q) = y_out(k, q) / sum_weights
    else:
        ! fallback: assign default or NaN
        for k = 1 to d:
            y_out(k, q) = y_ref(k, indices_used(q))  ! self value
```

3.4 Behavior and Flexibility

- The routine supports both scalar (e.g. $d = 1$) and vector-valued smoothing ($d > 1$).
- The use of `indices_used` allows smoothing over subsets without copying.
- The `x_query` values can be equal to, interpolated between, or distinct from `x_ref`.
- If `sum_weights = 0`, fallback to using the reference value at the current index (or an alternative default).

3.5 Applications Across TOX

- Normalization: LOESS fit for gene-wise SD vs. mean.
- Interpolation smoothing of expression vectors or shift vectors.
- Lexical smoothing across metadata axes.
- Clock hand and trajectory smoothing (e.g. Mjölnir-aligned).
- Local field smoothing in GAWD or Bobito cones.

3.6 Unit Tests for Generalized LOESS Smoothing (F42)

The following unit test cases validate the correct functioning of the generalized `loess_smooth` routine described above. Each test case assumes stateless design, preallocated input/output arrays, and strict reproducibility. No I/O, dynamic memory, or recursion is allowed.

Test Case 1: Constant Input (Sanity Check)

- **Input:**
  - `x_ref(i)` = 5.0 for all `i`
  - `y_ref(1, i)` = 10.0 for all `i`
  - `x_query(j)` = 5.0 for all `j`
  - `kernel_sigma` = 1.0
- **Expected Output:** `y_out(1, j)` = 10.0 for all `j`
- **Assertion:** `abs(y_out(1, j) - 10.0) < 1.0e-6`

Test Case 2: Linear Trend Recovery

- **Input:**
  - `x_ref(i)` = i
  - `y_ref(1, i)` = 0.5 * i
  - `x_query(j)` = j + 0.5
  - `kernel_sigma` = 2.0
- **Expected Output:** `y_out(1, j)` approximately follows `0.5 * x_query(j)`
- **Assertion:** `abs(y_out(1, j) - 0.5 * x_query(j)) < 0.05`

Test Case 3: Outlier Suppression

- **Input:**
  - `x_ref(1:10)` = 10.0, `x_ref(11)` = 100.0
  - `y_ref(1, 1:10)` = 5.0, `y_ref(1, 11)` = 99.0
  - `x_query(1)` = 10.0
  - `kernel_sigma` = 1.0, `kernel_cutoff` = 3.0
- **Expected Output:** Outlier at index 11 is ignored due to distance.
- **Assertion:** `abs(y_out(1,1) - 5.0) < 0.01`

Test Case 4: Sparse Fallback Behavior

- **Input:**
  - `x_ref(i)` = i * 100.0
  - `x_query(j)` = (j * 100.0) + 50.0
  - `kernel_sigma` = 1.0, very small
- **Expected Output:** Kernel weights are zero for all query points; fallback used.
- **Assertion:** `y_out(:, j)` equals `y_ref(:, indices_used(j))`

Test Case 5: Vector Field Smoothing

- **Input:**
  - `d = 3`, `y_ref(1:3, i)` = [1.0, 2.0, 3.0] + 0.1*i
  - `x_ref(i)` = i
  - `x_query(j)` = j + 0.5
- **Expected Output:** Each component smoothly follows its linear offset.
- **Assertion:** `abs(y_out(k, j) - expected(k, j)) < 0.1` for each `k`

3.7 The `which` Function for Logical Arrays (F42-Compliant)

The `which` function extracts the indices of all entries in a logical array that evaluate to `.TRUE.`. This routine is the direct analogue of the R function `which()`, designed to work with logical arrays resulting from any Fortran logical expression (e.g., `x > 3.0 .AND. MOD(x,2) == 0`). It follows the F42 principles: stateless, allocation-free, and usable in high-performance settings.

3.7.1 Interface Specification

```
subroutine which(
    mask(n),            ! Input logical array
    n,                  ! Size of the input array
    idx_out(m_max),     ! Preallocated output array of indices
    m_max,              ! Capacity of idx_out
    m_out               ! Number of matches returned
)
```

- `mask(n)`: Logical array derived from a Fortran logical expression
- `idx_out(m_max)`: Output array holding 1-based indices of matches
- `m_out`: Actual number of matches written to `idx_out`

3.7.2 Pseudocode Implementation

```
m_out = 0
do i = 1, n
    if (mask(i)) then
        if (m_out < m_max) then
            m_out = m_out + 1
            idx_out(m_out) = i
        endif
    endif
enddo
```

This implementation ensures no allocation and full compatibility with high-performance computing contexts (e.g., OpenMP, SIMD, or GPU-friendly flattening).

3.7.3 Test Case 1: Simple Threshold Filter

- **Input:** `x = [1.0, 4.0, 6.0, 2.0, 8.0]`
- **Logical Mask:** `mask = (x > 3.0)`
- **Expected Output:** `idx_out = [2, 3, 5]`, `m_out = 3`

3.7.4 Test Case 2: Composite Logical Condition

- **Input:** `x = [1.0, 4.0, 6.0, 2.0, 8.0]`
- **Mask:** `mask = (x > 3.0 .AND. MOD(INT(x),2) == 0)`
- **Expected Output:** `idx_out = [2, 5]`, `m_out = 2`

3.7.5 Test Case 3: No Matches

- **Input:** `x = [1.0, 2.0, 3.0]`
- **Mask:** `mask = (x > 10.0)`
- **Expected Output:** `idx_out = []`, `m_out = 0`

3.7.6 Performance and Compliance Notes

- This function is safe for use in any high-performance loop or parallel region.
- Overflow behavior (when `m_out` exceeds `m_max`) must be handled externally.
- All array bounds and allocations are managed by the caller.

4 F42-Compliant Index-Based K-D Trees and BSTs

4.1 Overview

This section defines efficient, allocation-free, recursion-free, and Fortran-compatible implementations of:

- A 1D Binary Search Tree (BST) represented as a flat index array
- A Cartesian K-D Tree using descending variance dimension order
- A Spherical K-D Tree (used for directional queries based on cosine similarity)
- Core access and query routines
- Unit tests for each structure

All memory structures are passed in. The trees are represented purely as sorted index arrays. No structs, recursion, or dynamic memory is used.

4.2 Binary Search Tree (BST) Construction via Sorting

Input:

- Real array `x(n)` — the data to be indexed
- Integer array `ix(n)` — index array to be filled

Pseudocode:

```
subroutine build_bst_index(x, n, ix)
  real, intent(in) :: x(n)
  integer, intent(out) :: ix(n)

  call quicksort_indices(x, ix, n)
end subroutine
```

4.3 Access Functions for BST (Flat Index Array)

Get Value at Sorted Position Returns the value of the original data array at the position given by the sorted index.

```
function get_sorted_value(x, ix, i) result(val)
real, intent(in) :: x(:)
integer, intent(in) :: ix(:)
integer, intent(in) :: i
real :: val

val = x(ix(i))
end function
```

Get Sorted Index for Range Query Example: Find values in `x` between a given `lo` and `hi`. This function returns a list of positions `i` such that `lo <= x(ix(i)) <= hi`.

```
subroutine bst_range_query(x, ix, n, lo, hi, out_ix, out_n)
real, intent(in) :: x(:)
integer, intent(in) :: ix(:)
integer, intent(in) :: n
real, intent(in) :: lo, hi
integer, intent(out) :: out_ix(n)
integer, intent(out) :: out_n

integer :: i
out_n = 0

do i = 1, n
if (x(ix(i)) >= lo .and. x(ix(i)) <= hi) then
out_n = out_n + 1
out_ix(out_n) = ix(i)
else if (x(ix(i)) > hi) then
exit ! Since array is sorted, we can stop early
end if
end do
end subroutine
```

Use Case Example Sort an array and extract all values between 2.0 and 3.5.

```
real :: x(1000)
integer :: ix(1000), res_ix(1000), res_n

call random_fill(x)
call build_bst_index(x, 1000, ix)
call bst_range_query(x, ix, 1000, 2.0, 3.5, res_ix, res_n)
! Now x(res_ix(1:res_n)) holds the sorted matching values
```

4.4 Cartesian K-D Tree Construction

Input:

- Real matrix `X(d, n)` — the data points
- Integer array `kd_ix(n)` — output index array representing K-D Tree

- Integer array `dim_order(d)` — precomputed dimension order by descending variance
- Integer work arrays for temporary use

Pseudocode:

```
subroutine build_kd_index(X, d, n, kd_ix, dim_order, work)
  ! Loop-based K-D Tree construction by sorting and partitioning
  ! No recursion. Uses stack to manage index intervals and split dimension.

  integer :: stack_top, stack(1:2, max_depth)
  integer :: l, r, dim, mid, depth, i

  stack_top = 1
  stack(:, 1) = [1, n]

  do while (stack_top > 0)
    l = stack(1, stack_top)
    r = stack(2, stack_top)
    stack_top = stack_top - 1
    if (r <= l) cycle

    depth = some_depth_heuristic(l, r)  ! Or simply count levels
    dim = dim_order(mod(depth, d) + 1)

    call partial_sort_by_dimension(X, d, kd_ix, l, r, dim)
  end do
end subroutine
```

4.5 Spherical K-D Tree for Directional Queries

Input:

- Normalized vector matrix `V(d, n)` — each column is unit length
- Integer array `sphere_ix(n)` — output index array for spherical K-D Tree

Pseudocode:

```
subroutine build_spherical_kd(V, d, n, sphere_ix, dim_order)
  call build_kd_index(V, d, n, sphere_ix, dim_order)
end subroutine
```

4.6 Access Routines

Get value at sorted position:

```
function get_value_sorted(x, ix, i)
  real :: get_value_sorted
  real, intent(in) :: x(:)
  integer, intent(in) :: ix(:)
  get_value_sorted = x(ix(i))
end function
```

Get point from KD index:

```
subroutine get_kd_point(X, kd_ix, i, point)
  real, intent(in) :: X(:, :)
  integer, intent(in) :: kd_ix(:)
  real, intent(out) :: point(:)
  point = X(:, kd_ix(i))
end subroutine
```

4.7 Unit Tests

BST Test:

```
call random_array(x, n)
call build_bst_index(x, n, ix)
assert_monotonic(x(ix))
```

K-D Tree Test:

```
call random_matrix(X, d, n)
call compute_dim_order(X, d, n, dim_order)
call build_kd_index(X, d, n, kd_ix, dim_order)
assert_valid_kd_structure(kd_ix)
```